

Implementación y análisis de complejidad de listas, pilas y colas en Java

Hayran Andrés López González

Objetivo del taller

Implementar las estructuras de datos List (listas enlazadas), Stack (pila) y Queue (cola) en Java, abarcando tanto arreglos dinámicos como listas enlazadas, realizando un análisis de complejidad de los métodos asociados.

Explicación de la implementación

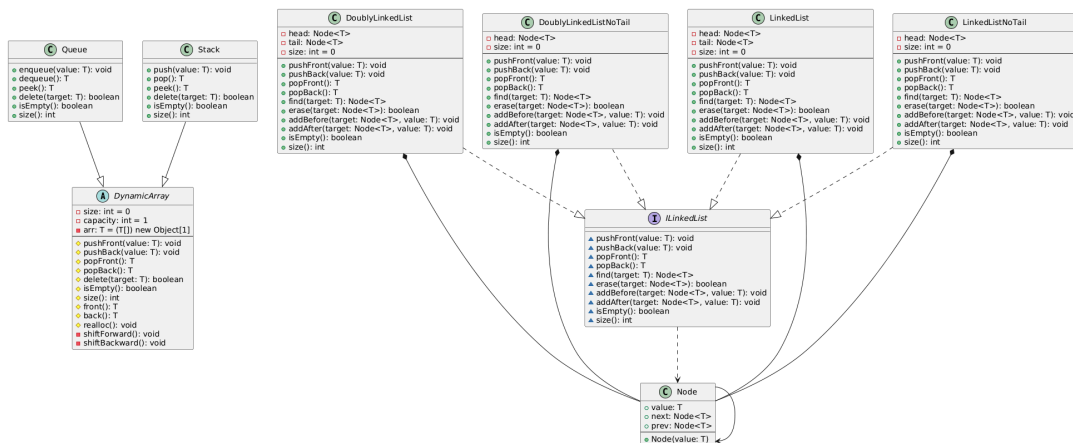
Para la implementación de este ejercicio se ha seguido la siguiente estructura:

- Una interfaz `ILinkedList` con los métodos comunes para todas las listas enlazadas (insertar, eliminar, buscar, etc.).
- Una clase abstracta `DynamicArray` con los métodos de un arreglo dinámico ya implementados. Es abstracta porque no la usaremos directamente.

De aquí parten las clases que si se usan:

- Las 4 variantes de listas enlazadas, todas implementando la interfaz `ILinkedList`:
 - ▶ `LinkedList`
 - ▶ `LinkedListNoTail`
 - ▶ `DoublyLinkedList`
 - ▶ `DoublyLinkedListNoTail`
- `Stack` y `Queue` para las pilas y colas respectivamente

El diagrama UML de este proyecto es el siguiente:



Análisis de complejidad

Listas enlazadas

Simple

- Buscar siempre toma $O(n)$, y es el cuello de botella de los siguientes métodos
- Insertar y eliminar al frente es inmediato porque tenemos un puntero a la cabeza de la lista y el siguiente nodo. Complejidad $O(1)$
- Insertar y eliminar al final requiere buscar el último nodo y el penúltimo (al eliminar), por lo que la complejidad es $O(n)$
- Añadir antes de un nodo requiere buscar el anterior ($O(n)$) pero añadir después se puede hacer de inmediato porque tenemos acceso al siguiente nodo ($O(1)$)
- Eliminar un nodo requiere buscar el anterior, entonces su complejidad es $O(n)$
- Verificar que está vacío es tan sencillo como ver si hay cabeza. Complejidad $O(1)$

Ten en cuenta que los métodos `addBefore` y `addAfter` ofrecen un puntero directo al nodo, entonces por ello pueden ser más rápidos.

Simple (con cola)

El nuevo puntero con cola hace que añadir elementos al final sea más sencillo porque tenemos la cola directamente. Pero el resto de operaciones siguen igual en cuanto requieren búsquedas (especialmente para buscar el nodo anterior a uno dado).

Dobles

Las listas enlazadas dobles introducen un puntero anterior que nos permite trabajar hacia atrás también. Gracias a eso, ahora borrar nodos y añadir antes de uno puede hacerse en tiempo constante porque podemos ver hacia atrás y cablear directamente.

Pero como no tienen puntero a cola, aún debemos buscarla, degradando las operaciones al final de la lista.

Dobles (con cola)

Ahora con el puntero a cola, podemos ir directo al final de la lista y operar sobre ella, haciendo que la adición/eliminación de elementos se torne de tiempo constante.

Con estas consideraciones, la tabla de complejidades queda como se ve a continuación. En azul las operaciones con rendimiento mejorado y en rojo las que quedan peor a comparación.

Operación	Simple	Simple (con cola)	Doble	Doble (con cola)
<code>pushFront</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>pushBack</code>	$O(n)$	$O(1)$	$O(n)$	$O(1)$
<code>popFront</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>popBack</code>	$O(n)$	$O(n)$	$O(n)$	$O(1)$
<code>find</code>	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<code>erase</code>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<code>addBefore</code>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<code>addAfter</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>isEmpty</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$

Comparando las tablas obtenemos que la mejor estructura en términos de rendimiento general es la **lista enlazada doble con puntero a cola**, al permitir un avance en dos direcciones y acceso a los dos extremos.

Arreglos dinámicos, pilas y colas

Los arreglos dinámicos son arreglos cuyo tamaño puede crecer de manera indefinida, permitiendo acceder a elementos de manera aleatoria (por índice) y reasignarse automáticamente cuando se llena.

Eliminar o añadir un elemento de una posición diferente a la última requiere trasladar el resto de elementos hacia adelante o hacia atrás, lo cual hace que su complejidad sea de $O(n)$. El resto de operaciones pueden hacerse de manera inmediata y por ende con $O(1)$.

La complejidad de sus operaciones es la siguiente:

Operación	Complejidad
pushFront	$O(n)$
pushBack	$O(1)$
popFront	$O(n)$
popBack	$O(1)$
delete	$O(n)$
isEmpty	$O(1)$
size	$O(1)$
front	$O(1)$
back	$O(1)$

Gráficas y resultados empíricos

Para analizar el comportamiento real de estos métodos utilizaremos medidas basadas en `java.time.Instant` y `java.time.Duration`. El esquema de medición es el siguiente:

- Mediremos el rendimiento con 10, 100, 1000, ..., hasta 10^{10} operaciones
- Esta medición es por cada método de cada clase implementada
- Haremos la tabla de tiempos y gráficas para los mismos. Una tabla por cada estructura de datos
- Las gráficas serán en escala logarítmica para evidenciar todas las escalas adecuadamente. Una gráfica por cada tabla

El código para registrar estos tiempos no interfiere con los algoritmos, y se ubica en la clase `Main`. Las gráficas se crean con **lilaq**, una librería de **Typst**, el mismo sistema con el que se ha compilado este PDF. los datos para lograrlo se registrarán en un CSV.

Listas enlazadas

Conclusiones